Twofold

IMPLEMENTATION PLAN

49 days to Twofold

One issue per day, from the first migration to a deployed demo. Every day says what to build, what you'll learn, and when it's done.

— Every birr, recorded twice.

2 Nov → 20 Dec 2026 | 7 milestones · 49 issues | Hosama Adem

49 days, **Mon 2 Nov → Sun 20 Dec 2026**. One issue per day, one branch per issue, one PR per branch. Requirements live in PRD.md; colours and components in design-system.md.

How to use this plan

1. Each morning, open the day's issue on GitHub (paste the block below as the body).
2. Create the branch named in the issue.
3. Read the **What you'll learn** list *before* coding; look up anything unfamiliar.
4. Ship only what **Done when** asks for. Extra ideas go in a new issue.
5. Open the PR with `Closes #<issue>`, wait for CI, merge, and write two lines in `docs/journal.md`: what you learned, what surprised you.

Sundays are for tests, review, and catching up. If a week slips, cut from the end of week 6 (QR) and week 7 (risk UI) first; never cut tests from weeks 2–3.

Milestones

Milestone	Week	Dates	Demo at the end
M1 Foundations	1	2–8 Nov	Sign up with OTP, log in, refresh tokens, see `/v1/me`
M2 Ledger core	2	9–15 Nov	Balanced journals, immutable ledger, idempotent API, property tests green
M3 Money movement	3	16–22 Nov	Send money with fees and limits, 1,000 concurrent transfers with no drift, OpenAPI in CI
M4 Async & integrity	4	23–29 Nov	Top up and cash out through a simulated provider with webhooks, drift monitor, BOLA suite
M5 Customer app	5	30 Nov–6 Dec	Real web app: sign up, home, send, top up, history, Playwright green
M6 Requests, QR & admin	6	7–13 Dec	Request money, pay by QR, admin console with MFA, maker-checker adjustments
M7 Hardening & launch	7	14–20 Dec	Risk engine, dashboards, load test report, deployed demo, case study

Labels

`backend` · `frontend` · `admin` · `ledger` · `security` · `infra` · `test` · `docs` · `good-first-read` (issues worth linking in a portfolio)

Calendar

Day	Date	Title
1	Mon 2 Nov	docs: rebrand to Twofold — PRD, plan, and design system
2	Tue 3 Nov	chore: reshape the backend for Twofold
3	Wed 4 Nov	feat: migrations with goose, queries with sqlc, and the users table
4	Thu 5 Nov	feat: problem+json errors, request IDs, and HTTP middleware
5	Fri 6 Nov	feat: sign up with phone number and OTP verification
6	Sat 7 Nov	feat: login with short-lived access tokens and rotating refresh tokens
7	Sun 8 Nov	feat: auth hardening — rate limits, sessions, and the audit log
8	Mon 9 Nov	feat: transaction PIN with lockout
9	Tue 10 Nov	feat: ledger schema — accounts, journals, and postings
10	Wed 11 Nov	feat: posting engine that only accepts balanced journals
11	Thu 12 Nov	feat: immutable ledger with reversals and cached balances
12	Fri 13 Nov	feat: wallets API with available and ledger balances

Day	Date	Title
13	Sat 14 Nov	feat: idempotency keys with request hashing and replay
14	Sun 15 Nov	test: property-based ledger tests — money is never created or lost
15	Mon 16 Nov	feat: peer-to-peer transfers with ordered row locking
16	Tue 17 Nov	test: concurrency suite — 1,000 parallel transfers, zero drift
17	Wed 18 Nov	feat: fee rules, transfer preview, and expected-fee check
18	Thu 19 Nov	feat: KYC tiers and transaction limits
19	Fri 20 Nov	feat: transaction history, receipts, and CSV export
20	Sat 21 Nov	feat: money requests
21	Sun 22 Nov	docs: OpenAPI 3.1 contract checked in CI
22	Mon 23 Nov	feat: provider interface and Telebirr/Chapa simulator
23	Tue 24 Nov	feat: top-ups confirmed by signed webhooks
24	Wed 25 Nov	feat: withdrawals with holds, payouts, and refunds
25	Thu 26 Nov	feat: transactional outbox and River workers
26	Fri 27 Nov	feat: in-app notifications from outbox events
27	Sat 28 Nov	feat: drift monitor and settlement reconciliation
28	Sun 29 Nov	test: BOLA authorization suite for every endpoint
29	Mon 30 Nov	feat(web): scaffold the customer app with Twofold design tokens
30	Tue 1 Dec	feat(web): generated API client and sign-up/login flows
31	Wed 2 Dec	feat(web): home screen with balance card and recent activity
32	Thu 3 Dec	feat(web): send money flow with fee preview and PIN
33	Fri 4 Dec	feat(web): top-up and cash-out flows with live status
34	Sat 5 Dec	feat(web): activity list, receipts, and statement export
35	Sun 6 Dec	test(web): Playwright end-to-end and accessibility checks
36	Mon 7 Dec	feat(web): request money and the requests inbox
37	Tue 8 Dec	feat: merchant profiles and QR payments
38	Wed 9 Dec	feat(web): profile, security settings, and KYC upload
39	Thu 10 Dec	feat(admin): staff login with TOTP and the console shell
40	Fri 11 Dec	feat(admin): user search, KYC review, and wallet freeze
41	Sat 12 Dec	feat(admin): transaction explorer and integrity dashboard
42	Sun 13 Dec	feat(admin): maker-checker adjustments and the audit log viewer
43	Mon 14 Dec	feat: risk rules engine — allow, flag, hold, block
44	Tue 15 Dec	feat(admin): risk flags queue
45	Wed 16 Dec	feat: observability with OpenTelemetry, Prometheus, and Grafana
46	Thu 17 Dec	perf: k6 load test and tuning report
47	Fri 18 Dec	security: threat model and ASVS level 1 review
48	Sat 19 Dec	chore: production images, deployment, and demo seed data
49	Sun 20 Dec	docs: launch — case study, ADR index, and demo video

Week 1 · M1 Foundations

The goal this week: a clean base and a real identity system. Nothing about money yet, on purpose — money code is only as safe as the auth in front of it.

Day 1 · Mon 2 Nov — `docs: rebrand to Twofold — PRD, plan, and design system`

Labels `docs` · **Milestone** M1 Foundations · **Branch** `docs/day-01-rebrand-to-twofold`

GITHUB ISSUE BODY

GOAL

Turn RealLedger into Twofold on paper before touching code: a detailed PRD, a day-by-day plan, a design system, and an archive of the old docs.

WHAT YOU'LL LEARN

- Writing a PRD that engineers can build from: personas, journeys, requirements with IDs, non-goals
- Writing down the ledger postings for every money flow before writing code
- Picking an accessible colour palette (WCAG contrast ratios)
- Crediting prior art correctly when a project has no licence

TASKS

- [ ] Move RealLedger docs to `docs/archive/realledger/` with a README explaining the pivot
- [ ] Write `docs/PRD.md` (product, ledger, concurrency, idempotency, state machines, risk, data model, API, NFRs, testing)
- [ ] Write `docs/implementation-plan.md` (49 days, every issue)
- [ ] Write `docs/design-system.md` and `docs/brand/palette.html`
- [ ] Rewrite the root `README.md` for Twofold

DONE WHEN

- [ ] Every PRD section links to the day that builds it
- [ ] Palette contrast pairs meet WCAG AA
- [ ] `make up && make run` still works (no code changed)

Day 2 · Tue 3 Nov — `chore: reshape the backend for Twofold`

Labels `backend` `infra` · **Milestone** M1 · **Branch** `chore/day-02-reshape-backend`

GITHUB ISSUE BODY

GOAL

Rename the Go module, Docker names, and env vars to Twofold, and create the package layout from PRD §16.2 so every later feature has an obvious home.

WHAT YOU'LL LEARN

- Go module paths and how a rename ripples through imports
- Package-by-feature layout vs package-by-layer, and why `internal/` matters
- Renaming Docker volumes without losing data (or deciding to start fresh)

TASKS

- [ ] Module path `github.com/hosama-adem/twofold/backend`; update imports
- [ ] Compose: container `twofold-db`, volume `twofold-pgdata`, database `twofold`
- [ ] `.env.example`: `DATABASE_URL` for `twofold`, replace tenant/platform JWT secrets with `JWT_SIGNING_KEY` and `STAFF_JWT_SIGNING_KEY` placeholders
- [ ] Create `internal/{platform,auth,user,ledger,wallet,transfer,fee,limit,risk,funding,provider,request,merchant,idempo` each with a `doc.go` stating its job in one sentence
- [ ] Move config, database, and httpserver under `internal/platform/`
- [ ] Rename the GitHub repo to `twofold` (Settings → General); update the remote

DONE WHEN

- [ ] `go build ./...`, `go vet ./...`, `go test ./...`, and lint pass
- [ ] `make up && make run` → `/health` 200 and `/ready` 200
- [ ] No `realledger` string left outside `docs/archive/` (`rg -i realledger`)

Day 3 · Wed 4 Nov — `feat: migrations with goose, queries with sqlc, and the users table`

Labels `backend` · **Milestone** M1 · **Branch** `feat/day-03-migrations-sqlc-users`

GITHUB ISSUE BODY

GOAL

A repeatable way to change the schema and type-safe SQL, starting with `users` and `otp_codes`.

WHAT YOU'LL LEARN

- Versioned migrations: up/down, why you never edit a merged migration
- sqlc: write SQL, get typed Go; why not an ORM for money code
- Postgres constraints as the last line of defence (unique, check, not null)
- `citext`, `timestamptz`, UUIDv7 and why time-ordered IDs help indexes

TASKS

- [ ] Add goose; `make migrate-up`, `make migrate-down`, `make migrate-new name=...`
- [ ] Migration: `users` and `otp_codes` per PRD §14.3, with check constraints (E.164 phone format, `kyc_tier between 0 and 2`)
- [ ] `sqlc.yaml` + `internal/user/queries.sql` (create, get by phone, get by id, mark phone verified)
- [ ] `make generate` runs sqlc; CI fails if generated code is stale
- [ ] Integration test helper using testcontainers-go (fresh Postgres per package)

DONE WHEN

- [ ] `make migrate-up && make migrate-down && make migrate-up` is clean
- [ ] Repository test proves duplicate phone numbers are rejected by the database
- [ ] CI runs integration tests

Day 4 · Thu 5 Nov — `feat: problem+json errors, request IDs, and HTTP middleware`

Labels `backend` · **Milestone** M1 · **Branch** `feat/day-04-errors-middleware`

GITHUB ISSUE BODY

GOAL

Every response — success or failure — is predictable, traceable, and safe.

WHAT YOU'LL LEARN

- RFC 9457 problem details and stable machine-readable error codes
- Separating domain errors from transport errors in Go (`errors.Is/As`)
- Middleware order: request ID → recover → logging → timeout → auth
- Why you never leak SQL or stack traces to clients

TASKS

- [ ] `internal/platform/httpx` : `Problem` type, `WriteError(c, err)` mapping domain errors to status + code
- [ ] Request ID middleware (accept `X-Request-ID` or generate, echo it back, put it in logs)
- [ ] Recover middleware returning a problem response, logging the panic
- [ ] Access log middleware (method, route template, status, duration, request ID) — no bodies
- [ ] Body size limit (1 MB) and request timeout (10 s)
- [ ] JSON decoding that rejects unknown fields

DONE WHEN

- [ ] Table tests cover 400, 404, 409, 413, 422, 500 shapes
- [ ] A panic in a handler returns problem+json with the request ID and no stack trace

Day 5 · Fri 6 Nov — `feat: sign up with phone number and OTP verification`

Labels `backend` `security` · **Milestone** M1 · **Branch** `feat/day-05-register-otp`

GITHUB ISSUE BODY

GOAL

`POST /v1/auth/register` and `POST /v1/auth/verify-phone` (FR-ID-1..3).

WHAT YOU'LL LEARN

- Password hashing with argon2id and choosing its parameters
- OTP design: hash the code, limit attempts, expire fast, single use
- Avoiding account enumeration in responses
- Interfaces for side effects (`SMSSender`) so tests stay fast

TASKS

- [ ] Validate phone (E.164, Ethiopian `+2519…` / `+2517…`), password policy, display name
- [ ] Store argon2id hash; create user in `pending_verification`
- [ ] Generate 6-digit OTP with `crypto/rand` , store SHA-256 hash, 5-minute expiry, max 5 attempts
- [ ] `SMSSender` interface; dev implementation logs the code
- [ ] Verify endpoint marks phone verified (wallet creation joins later on Day 12)
- [ ] Resend with cooldown (60 s)

DONE WHEN

- [ ] Tests: happy path, wrong code, expired code, too many attempts, reused code
- [ ] Register with an existing phone returns the same response shape as a new one

Day 6 · Sat 7 Nov — `feat: login with short-lived access tokens and rotating refresh tokens`

Labels `backend` `security` · **Milestone** M1 · **Branch** `feat/day-06-login-refresh`

GITHUB ISSUE BODY

GOAL

`/v1/auth/login`, `/v1/auth/refresh`, `/v1/auth/logout`, `GET /v1/me` (FR-ID-4..6).

WHAT YOU'LL LEARN

- Access vs refresh tokens, and why access tokens live 15 minutes
- Refresh token rotation and reuse detection (token families)
- Ed25519 (EdDSA) JWT signing and the `kid` header for key rotation
- Constant-time comparison and timing attacks on login

TASKS

- [ ] JWT access token (15 min) with `sub`, `sid`, `kid`; EdDSA keys from env
- [ ] Opaque refresh token (30 days), only its hash stored in `sessions`
- [ ] Refresh: rotate, mark old as replaced; reuse of an old token revokes the whole family
- [ ] Auth middleware puts `UserID` and `SessionID` in context
- [ ] `GET /v1/me` returns profile and tier

DONE WHEN

- [ ] Test: reusing a rotated refresh token revokes every session in the family
- [ ] Test: expired and tampered tokens return 401 problem+json

Day 7 · Sun 8 Nov — `feat: auth hardening — rate limits, sessions, and the audit log`

Labels `backend` `security` · **Milestone** M1 · **Branch** `feat/day-07-auth-hardening-audit`

GITHUB ISSUE BODY

GOAL

Close the week by making auth abuse-resistant and recording security events.

WHAT YOU'LL LEARN

- Token-bucket rate limiting per IP and per phone number
- Append-only tables enforced by triggers
- What belongs in an audit log (who, what, target, before/after, request ID)

TASKS

- [ ] Rate limits on register, verify, login (in-memory now, interface for Redis later)
- [ ] `GET /v1/me/sessions`, `DELETE /v1/me/sessions/{id}`
- [ ] `audit_logs` migration with a trigger that rejects UPDATE and DELETE
- [ ] `audit.Record(ctx, tx, entry)` helper, used for login success/failure, session revoke
- [ ] Week 1 review: update README, write `docs/journal.md` entry, demo sign-up → login → me

DONE WHEN

- [ ] Test: 6th login attempt within a minute returns 429 with `Retry-After`
- [ ] Test: updating an audit row fails at the database

Week 2 · M2 Ledger core

The heart of Twofold. Every later feature is a thin layer over what you build this week, so it gets the most tests.

Day 8 · Mon 9 Nov — `feat: transaction PIN with lockout`

Labels `backend` `security` · **Milestone** M2 Ledger core · **Branch** `feat/day-08-transaction-pin`

GITHUB ISSUE BODY

GOAL

A 4-digit PIN required for every money movement (FR-ID-7).

WHAT YOU'LL LEARN

- Step-up authentication: why a PIN on top of a session
- Lockouts that resist brute force without enabling denial of service
- Updating counters safely under concurrency (`UPDATE … RETURNING`)

TASKS

- [ ] `PUT /v1/me/pin` (set requires password; change requires old PIN)
- [ ] Reject weak PINs (`0000` , `1234` , repeated, sequential)
- [ ] argon2id hash; 5 failures → locked 15 minutes; counter resets on success
- [ ] `pin.Verify(ctx, userID, pin)` for later handlers; audit failures and lockouts

DONE WHEN

- [ ] Tests: weak PIN rejected, lockout after 5 failures, unlock after expiry
- [ ] Wrong-PIN response tells the user how many tries are left

Day 9 · Tue 10 Nov — `feat: ledger schema — accounts, journals, and postings`

Labels `backend` `ledger` `good-first-read` · **Milestone** M2 · **Branch** `feat/day-09-ledger-schema`

GITHUB ISSUE BODY

GOAL

The tables from PRD §7 and §14 with the database enforcing as much of the accounting as it can.

WHAT YOU'LL LEARN

- Double-entry: assets vs liabilities, normal balances, debit/credit direction
- Why wallets are liabilities on Twofold's books
- Encoding invariants as constraints: `amount_minor > 0` , currency match, enums
- Seeding system accounts in a migration

TASKS

- [ ] Migration: `accounts` , `journal_entries` , `postings` , `transactions` , `transaction_events`
- [ ] Checks: positive amounts, valid direction, `allow_negative` false by default
- [ ] Seed system accounts: `provider_clearing:telebirr` , `provider_clearing:chapa` , `payouts_pending` , `fee_revenue` , `suspense`
- [ ] Write `docs/adr/0001-double-entry-ledger.md` in your own words

DONE WHEN

- [ ] Inserting a zero or negative posting fails at the database
- [ ] ADR explains direction + positive amount vs signed amounts

Day 10 · Wed 11 Nov — `feat: posting engine that only accepts balanced journals`

Labels `backend` `ledger` · **Milestone** M2 · **Branch** `feat/day-10-posting-engine`

GITHUB ISSUE BODY

GOAL

`ledger.Post(ctx, tx, Journal)` — the only function in the codebase allowed to move money.

WHAT YOU'LL LEARN

- Validating a journal: at least two lines, one currency, debits equal credits
- Computing a balance change from direction and the account's normal balance
- Writing an API that is hard to misuse (unexported constructors, value types)

TASKS

- [ ] `ledger.Journal`, `ledger.Line`, `ledger.Money` (int64 minor + currency)
- [ ] Validation errors: `ErrUnbalanced`, `ErrCurrencyMismatch`, `ErrEmptyJournal`
- [ ] Insert journal + postings in the caller's transaction
- [ ] Deferred constraint trigger: every journal's debits equal credits at commit

DONE WHEN

- [ ] Unit tests for each validation error
- [ ] Integration test: an unbalanced journal inserted with raw SQL fails at COMMIT

Day 11 · Thu 12 Nov — `feat: immutable ledger with reversals and cached balances`

Labels `backend` `ledger` · **Milestone** M2 · **Branch** `feat/day-11-immutability-balances`

GITHUB ISSUE BODY

GOAL

History can never be edited, only reversed; balances are fast to read and always provable.

WHAT YOU'LL LEARN

- Why financial records are append-only
- Reversal journals and the "reverse once" unique constraint
- Cached balance + version column vs computing from postings
- Optimistic versioning as a second guard

TASKS

- [ ] Triggers rejecting UPDATE/DELETE on `journal_entries` and `postings`
- [ ] `ledger.Reverse(ctx, tx, journalID, reason)`; unique `reverses_journal_id`
- [ ] `Post` updates `accounts.balance_minor` and `version`, writes `balance_after_minor` on each posting
- [ ] Non-negative check for accounts with `allow_negative = false`
- [ ] Query: recompute a balance from postings (used by the drift monitor on Day 27)

DONE WHEN

- [ ] Test: cached balance equals recomputed balance after 100 random journals
- [ ] Test: reversing twice fails; UPDATE on postings fails

Day 12 · Fri 13 Nov — `feat: wallets API with available and ledger balances`

Labels `backend` · **Milestone** M2 · **Branch** `feat/day-12-wallets`

GITHUB ISSUE BODY

GOAL

Each verified user gets an ETB wallet; `GET /v1/wallets` shows what they can spend (FR-WAL).

WHAT YOU'LL LEARN

- Ledger balance vs available balance (holds)
- Creating related rows atomically in one transaction
- Freeze modes and where to enforce them

TASKS

- [ ] Migration: `wallets` (status, freeze_mode, frozen_reason)
- [ ] Phone verification creates the liability account + wallet in the same DB transaction
- [ ] Holds: available = ledger − pending withdrawals (column or view)
- [ ] `GET /v1/wallets` , `GET /v1/recipients/lookup?q=` with masked names
- [ ] Dev-only `make seed` that funds a test wallet through a proper journal

DONE WHEN

- [ ] Test: verify phone creates exactly one wallet even if called twice concurrently
- [ ] Lookup never returns the full phone number

Day 13 · Sat 14 Nov — `feat: idempotency keys with request hashing and replay`

Labels `backend` `good-first-read` · **Milestone** M2 · **Branch** `feat/day-13-idempotency`

GITHUB ISSUE BODY

GOAL

Retrying a money request never moves money twice (PRD §9).

WHAT YOU'LL LEARN

- Why networks make retries inevitable
- Key + request hash + stored response; 409 for in-flight, 422 for a different body
- Doing the claim and the business write in one database transaction
- Cleanup of expired keys

TASKS

- [ ] Migration: `idempotency_keys` (user_id, key) primary key
- [ ] Middleware for routes marked idempotent: require header, canonical JSON hash, claim row
- [ ] Replay stored status + body with `Idempotent-Replayed: true`
- [ ] 24-hour expiry; cleanup query (job arrives Day 25)

DONE WHEN

- [ ] Tests: same key + same body replays; same key + new body → 422; concurrent same key → one 409
- [ ] Write `docs/adr/0002-idempotency.md`

Day 14 · Sun 15 Nov — `test: property-based ledger tests — money is never created or lost`

Labels `test` `ledger` · **Milestone** M2 · **Branch** `test/day-14-ledger-properties`

GITHUB ISSUE BODY

GOAL

Let the computer try thousands of random journals and prove the invariants hold.

WHAT YOU'LL LEARN

- Property-based testing with `pgregory.net/rapid`
- Thinking in invariants rather than examples
- Shrinking: how a failing case is reduced to the smallest example

TASKS

- [ ] Generators for valid and invalid journals
- [ ] Invariant: sum of all debits = sum of all credits, always
- [ ] Invariant: every cached balance equals its recomputed balance
- [ ] Invariant: no non-negative account goes below zero
- [ ] Invariant: reverse(post(j)) leaves every balance unchanged
- [ ] Week 2 review and journal entry

DONE WHEN

- [ ] Properties run 1,000 cases in CI under 60 s
- [ ] README links the property tests as "how we know the ledger is correct"

Week 3 · M3 Money movement

Day 15 · Mon 16 Nov — `feat: peer-to-peer transfers with ordered row locking`

Labels `backend` `ledger` `good-first-read` · **Milestone** M3 Money movement · **Branch** `feat/day-15-p2p-transfers`

GITHUB ISSUE BODY

GOAL

`POST /v1/transfers` moves money between two users atomically (FR-TRF).

WHAT YOU'LL LEARN

- Lost updates and why `SELECT … FOR UPDATE` prevents them
- Deadlocks, and preventing them by locking rows in sorted ID order
- Why hot system accounts (fee revenue) are not locked
- Transaction state machine: created → completed / failed

TASKS

- [ ] Handler: auth, PIN, idempotency, validate amount and recipient
- [ ] Service: one DB transaction — lock both wallets in ID order, check status and available balance, post journal, insert `transactions` + `transaction_events`
- [ ] Human-friendly reference (`TF-7K2M-9QXA`)
- [ ] Errors: insufficient funds, self-transfer, frozen wallet, recipient not found

DONE WHEN

- [ ] Tests for each error and the happy path
- [ ] Journal for a transfer matches PRD §7.3 exactly

Day 16 · Tue 17 Nov — `test: concurrency suite — 1,000 parallel transfers, zero drift`

Labels `test` `ledger` · **Milestone** M3 · **Branch** `test/day-16-concurrency-suite`

GITHUB ISSUE BODY

GOAL

Prove the transfer path is safe under heavy contention.

WHAT YOU'LL LEARN

- Writing deterministic concurrency tests with goroutines and barriers
- Detecting deadlocks (Postgres error 40P01) and serialization failures
- The Go race detector

TASKS

- [ ] A↔B opposing transfers × 500 each in parallel: no deadlock, totals conserved
- [ ] Ten senders draining one wallet: never negative, exact number succeed
- [ ] Same idempotency key from 50 goroutines: exactly one transfer
- [ ] Run with `-race` ; add `make test-concurrency`
- [ ] Write `docs/adr/0003-lock-ordering.md`

DONE WHEN

- [ ] Suite passes 20 runs in a row locally and in CI

Day 17 · Wed 18 Nov — `feat: fee rules, transfer preview, and expected-fee check`

Labels `backend` · **Milestone** M3 · **Branch** `feat/day-17-fees-preview`

GITHUB ISSUE BODY

GOAL

Fees are configurable, integer-exact, and always shown before confirming (FR-FEE).

WHAT YOU'LL LEARN

- Basis points and integer rounding rules (round half up, in minor units)
- Effective-dated configuration
- Why the client sends the fee it saw (`expected_fee_minor`) and the server rejects a mismatch

TASKS

- [ ] Migration + seed `fee_rules` (flat, rate_bps, min, max, per kind and tier)
- [ ] `fee.Calculate(kind, tier, amount)` with table tests including rounding edges
- [ ] `POST /v1/transfers/preview` returns amount, fee, total, recipient display name
- [ ] Transfers post the fee line to `fee_revenue` ; reject if expected fee differs (409 `fee_changed`)

DONE WHEN

- [ ] Property: fee never exceeds max, never below min, never negative

Day 18 · Thu 19 Nov — `feat: KYC tiers and transaction limits`

Labels `backend` `security` · **Milestone** M3 · **Branch** `feat/day-18-kyc-limits`

GITHUB ISSUE BODY

GOAL

Limits from PRD §12.1 enforced on every money movement; users can submit KYC (FR-KYC).

WHAT YOU'LL LEARN

- Why regulators require tiered KYC
- Rolling daily/monthly windows with efficient SQL
- Checking limits inside the money transaction to avoid race conditions

TASKS

- [ ] Migration + seed `tier_limits` ; `kyc_submissions`
- [ ] `limit.Check(ctx, tx, user, kind, amount)` — balance cap, per-transaction, daily, monthly
- [ ] `POST /v1/kyc/submissions` (data + document keys; files arrive Day 38)
- [ ] `GET /v1/me` shows limit usage
- [ ] Clear errors: `limit_exceeded` with which limit and how much remains

DONE WHEN

- [ ] Tests at each boundary (limit − 1, limit, limit + 1)
- [ ] Recipient balance cap is enforced too

Day 19 · Fri 20 Nov — `feat: transaction history, receipts, and CSV export`

Labels `backend` · **Milestone** M3 · **Branch** `feat/day-19-history`

GITHUB ISSUE BODY

GOAL

Users can see, search, and export everything that happened (FR-HIS).

WHAT YOU'LL LEARN

- Keyset (cursor) pagination vs offset, and why offset breaks at scale
- Composite indexes that match your query
- Streaming CSV without loading everything into memory
- CSV injection and how to prevent it

TASKS

- [ ] `GET /v1/transactions?cursor=&kind=&status=&from=&to=` (newest first)
- [ ] `GET /v1/transactions/{reference}` receipt (only for participants)
- [ ] `GET /v1/transactions/export.csv` streamed, with `=+-@` prefix escaping
- [ ] `EXPLAIN ANALYZE` the main query; add the index; paste the plan in the PR

DONE WHEN

- [ ] Pagination test: no duplicates or gaps while new transactions arrive

Day 20 · Sat 21 Nov — `feat: money requests`

Labels `backend` · **Milestone** M3 · **Branch** `feat/day-20-money-requests`

GITHUB ISSUE BODY

GOAL

Ask someone for money; they pay, decline, or let it expire (FR-REQ, PRD §10.4).

WHAT YOU'LL LEARN

- Modelling a state machine with guarded transitions in SQL (`UPDATE … WHERE status = 'pending'`)
- Reusing the transfer service instead of duplicating money logic
- A unique constraint so a request can only be paid once

TASKS

- [ ] Migration `money_requests`
- [ ] Create, list (incoming/outgoing), pay (calls transfer service), decline, cancel
- [ ] Expiry after 7 days (query now, job on Day 25)

DONE WHEN

- [ ] Test: paying the same request twice concurrently moves money once

Day 21 · Sun 22 Nov — `docs: OpenAPI 3.1 contract checked in CI`

Labels `docs` `test` · **Milestone** M3 · **Branch** `docs/day-21-openapi-contract`

GITHUB ISSUE BODY

GOAL

`docs/openapi.yaml` describes every customer endpoint, and CI fails when the server drifts from it.

WHAT YOU'LL LEARN

- Contract-first vs code-first APIs
- Describing problem+json errors and money fields in OpenAPI
- Contract tests: validating real responses against the schema

TASKS

- [ ] Write the spec (schemas, examples, error responses, `Idempotency-Key` header)
- [ ] Lint with Redocly in CI
- [ ] Integration tests validate responses with kin-openapi
- [ ] Serve docs at `/docs` in dev
- [ ] Week 3 review: demo "send money with fee and limits"; journal entry

DONE WHEN

- [ ] Changing a response field without updating the spec fails CI

Week 4 · M4 Async & integrity

Day 22 · Mon 23 Nov — `feat: provider interface and Telebirr/Chapa simulator`

Labels `backend` · **Milestone** M4 Async & integrity · **Branch** `feat/day-22-provider-simulator`

GITHUB ISSUE BODY

GOAL

A `Provider` interface (PRD §11.4) and a simulator that behaves like a real, slightly unreliable payment provider.

WHAT YOU'LL LEARN

- Ports and adapters: keeping provider details out of the core
- Simulating failure modes: delays, declines, duplicate and out-of-order webhooks
- HMAC-SHA256 signatures with timestamps to stop replay

TASKS

- [ ] `provider.Provider` with `InitiateCollection`, `InitiatePayout`, `GetStatus`, `VerifyWebhook`
- [ ] `cmd/simulator`: small HTTP service, scenarios via amount suffix (e.g. `.13` → decline)
- [ ] Simulator signs webhooks; Twofold verifies signature + 5-minute timestamp window
- [ ] Add simulator to docker compose

DONE WHEN

- [ ] Unit tests for signature verification (valid, tampered, expired)

Day 23 · Tue 24 Nov — `feat: top-ups confirmed by signed webhooks`

Labels `backend` `ledger` · **Milestone** M4 · **Branch** `feat/day-23-topups-webhooks`

GITHUB ISSUE BODY

GOAL

Add money from Telebirr/Chapa; the wallet is credited only when the provider confirms (FR-TOP, PRD §10.2).

WHAT YOU'LL LEARN

- Why "pending" is a real state in payments
- Webhook dedupe with `(provider, event_id)` primary key
- Handling out-of-order events with state machine guards
- Polling as a fallback when webhooks never arrive

TASKS

- [ ] `POST /v1/topups` (idempotent) creates transaction + `provider_payments`, returns checkout info
- [ ] `POST /webhooks/providers/{provider}`: verify, dedupe, transition, post journal (clearing → wallet)
- [ ] Late success after failure goes to `suspense` and raises an alert
- [ ] `GET /v1/topups/{id}`

DONE WHEN

- [ ] Tests: duplicate webhook credits once; out-of-order events end in the right state

Day 24 · Wed 25 Nov — `feat: withdrawals with holds, payouts, and refunds`

Labels `backend` `ledger` · **Milestone** M4 · **Branch** `feat/day-24-withdrawals`

GITHUB ISSUE BODY

GOAL

Cash out to a saved destination; money is held, paid out, or refunded (FR-WDR, PRD §10.3).

WHAT YOU'LL LEARN

- Two-phase money movement: reserve, then settle or release
- Encrypting sensitive fields at rest (AES-GCM) and masking on read
- Compensating actions instead of rollbacks across systems

TASKS

- [ ] `payout_destinations` (encrypted identifier) + endpoints
- [ ] `POST /v1/withdrawals`: PIN, limits, tier check, journal wallet → `payouts_pending`
- [ ] Payout success: `payouts_pending` → clearing; failure: refund journal back to wallet
- [ ] Notify-ready events for each outcome

DONE WHEN

- [ ] Test: failed payout restores the exact available balance

Day 25 · Thu 26 Nov — `feat: transactional outbox and River workers`

Labels `backend` `good-first-read` · **Milestone** M4 · **Branch** `feat/day-25-outbox-river`

GITHUB ISSUE BODY

GOAL

Side effects happen reliably after commit, never before and never lost (PRD §11.1–11.2).

WHAT YOU'LL LEARN

- The dual-write problem and the transactional outbox pattern
- At-least-once delivery and why consumers must be idempotent
- Postgres-backed job queues (River), retries with backoff, dead letters

TASKS

- [ ] `outbox_events` written in the same transaction as business changes
- [ ] Dispatcher uses `FOR UPDATE SKIP LOCKED` to publish to River
- [ ] Jobs: idempotency key cleanup, request expiry, top-up status polling
- [ ] `cmd/worker` binary; `make worker`
- [ ] Write `docs/adr/0004-outbox.md`

DONE WHEN

- [ ] Test: crash between commit and dispatch still delivers the event

Day 26 · Fri 27 Nov — `feat: in-app notifications from outbox events`

Labels `backend` · **Milestone** M4 · **Branch** `feat/day-26-notifications`

GITHUB ISSUE BODY

GOAL

Users get a notification for every money event (FR-NOT).

WHAT YOU'LL LEARN

- Idempotent consumers (dedupe by event ID)
- Templating user-facing copy separately from logic
- Server-Sent Events for live updates (optional stretch)

TASKS

- [ ] `notifications` table; consumer turns events into notifications
- [ ] `GET /v1/notifications` , mark read
- [ ] SMS sender interface reused for critical events (dev logs)

DONE WHEN

- [ ] Test: the same event processed twice creates one notification

Day 27 · Sat 28 Nov — `feat: drift monitor and settlement reconciliation`

Labels `backend` `ledger` · **Milestone** M4 · **Branch** `feat/day-27-integrity-jobs`

GITHUB ISSUE BODY

GOAL

Twofold checks itself (PRD §11.5).

WHAT YOU'LL LEARN

- Reconciliation: internal books vs internal books, and internal vs provider
- Suspense accounts and how operations teams use them
- Reporting problems without auto-"fixing" money

TASKS

- [ ] Drift job: cached vs recomputed balance for every account; global debits = credits
- [ ] Settlement job: compare simulator's daily report with `provider_payments`
- [ ] `integrity_reports` table; metrics `twofold_ledger_drift_accounts`
- [ ] Mismatches open a report, never edit balances

DONE WHEN

- [ ] Test: a manually corrupted cached balance is detected and reported

Day 28 · Sun 29 Nov — `test: BOLA authorization suite for every endpoint`

Labels `test` `security` · **Milestone** M4 · **Branch** `test/day-28-bola-suite`

GITHUB ISSUE BODY

GOAL

Prove no user can read or act on another user's data (OWASP API1).

WHAT YOU'LL LEARN

- Broken Object Level Authorization, the top API vulnerability
- Table-driven security tests generated from the route list
- Returning 404 instead of 403 to avoid leaking existence

TASKS

- [ ] Two users, Alice and Bob, with data of every kind
- [ ] For every route with an ID, Bob tries Alice's ID → 404
- [ ] Test fails if a new route is added without a BOLA case
- [ ] Week 4 review: demo top-up → transfer → cash-out; journal entry

DONE WHEN

- [ ] Suite covers 100% of ID-bearing routes

Week 5 · M5 Customer app

Use the colours, type, and money rules from design-system.md. Mobile-first, 480 px max width.

Day 29 · Mon 30 Nov — `feat(web): scaffold the customer app with Twofold design tokens`

Labels `frontend` · **Milestone** M5 Customer app · **Branch** `feat/day-29-web-scaffold`

GITHUB ISSUE BODY

GOAL

`apps/web` running with the design system wired in.

WHAT YOU'LL LEARN

- Next.js App Router layout and route groups
- Design tokens as CSS variables + Tailwind theme
- shadcn/ui: owning your components instead of importing a library
- Light and dark mode without flash

TASKS

- [ ] Next.js + TypeScript (strict) + Tailwind + shadcn/ui + pnpm
- [ ] Tokens from design-system §9; fonts (Plus Jakarta Sans, Inter, Noto Sans Ethiopic)
- [ ] `formatMoney(minor, currency)` + `parseAmount(input)` with Vitest tests
- [ ] App shell: top bar, bottom nav (Home, Activity, Send, Profile)
- [ ] CI job: typecheck, lint, test, build

DONE WHEN

- [ ] `formatMoney(10200, "ETB")` → `ETB 102.00`; parse rejects 3 decimals
- [ ] Lighthouse accessibility ≥ 95 on the shell

Day 30 · Tue 1 Dec — `feat(web): generated API client and sign-up/login flows`

Labels `frontend` · **Milestone** M5 · **Branch** `feat/day-30-web-auth`

GITHUB ISSUE BODY

GOAL

Users can sign up, verify their phone, set a PIN, and log in from the browser.

WHAT YOU'LL LEARN

- Generating a typed client from OpenAPI (openapi-typescript + openapi-fetch)
- Where to keep tokens in a browser (httpOnly cookie via a BFF route vs memory)
- Form validation with react-hook-form + zod; mapping problem+json to field errors

TASKS

- [ ] `pnpm gen:api`; CI fails if generated types are stale
- [ ] Screens: welcome, register, OTP (6 boxes, paste support, resend timer), set PIN, login
- [ ] Refresh tokens through a Next.js route handler with httpOnly cookies
- [ ] TanStack Query provider; 401 → refresh once → retry

DONE WHEN

- [ ] Full sign-up works against the local API

Day 31 · Wed 2 Dec — `feat(web): home screen with balance card and recent activity`

Labels `frontend` · **Milestone** M5 · **Branch** `feat/day-31-web-home`

GITHUB ISSUE BODY

GOAL

The signature screen: gradient balance card, quick actions, recent activity.

WHAT YOU'LL LEARN

- Loading, empty, and error states as first-class designs
- Skeletons that match final layout (no layout shift)
- Hiding sensitive numbers accessibly

TASKS

- [ ] Balance card (available, ledger, holds) with hide toggle remembered locally
- [ ] Quick actions: Send, Request, Top up, Cash out, Scan
- [ ] Recent 5 transactions with money rules (credit green +, debit −, pending, failed)
- [ ] Limits usage bar from `/v1/me`

DONE WHEN

- [ ] Matches `docs/brand/palette.html` sketch; screen reader reads amounts as words

Day 32 · Thu 3 Dec — `feat(web): send money flow with fee preview and PIN`

Labels `frontend` `good-first-read` · **Milestone** M5 · **Branch** `feat/day-32-web-send`

GITHUB ISSUE BODY

GOAL

Recipient → amount → review → PIN → success, in under 30 seconds.

WHAT YOU'LL LEARN

- Multi-step flows with URL state (back button works)
- Generating the idempotency key once per attempt, not per click
- Optimistic UI vs waiting for the server — and why money waits

TASKS

- [ ] Recipient search (phone/handle) with masked result
- [ ] Amount keypad; preview call shows fee and total
- [ ] PIN sheet; send with `Idempotency-Key` and `expected_fee_minor`
- [ ] Success screen with receipt link; clear error screens for each problem code

DONE WHEN

- [ ] Double-tapping "Send" creates one transfer

Day 33 · Fri 4 Dec — `feat(web): top-up and cash-out flows with live status`

Labels `frontend` · **Milestone** M5 · **Branch** `feat/day-33-web-topup-cashout`

GITHUB ISSUE BODY

GOAL

Async money movements that never leave the user guessing.

WHAT YOU'LL LEARN

- Polling with backoff (or SSE) and stopping when final
- Status timelines as a UX pattern for pending money

TASKS

- [ ] Top-up: provider choice, amount, redirect to simulator checkout, return to status timeline
- [ ] Cash-out: choose/add destination, amount, PIN, status timeline
- [ ] Copy from design-system §8 for every state

DONE WHEN

- [ ] Simulator decline shows a clear failed state and the refunded balance

Day 34 · Sat 5 Dec — `feat(web): activity list, receipts, and statement export`

Labels `frontend` · **Milestone** M5 · **Branch** `feat/day-34-web-activity`

GITHUB ISSUE BODY

GOAL

Full history with filters, receipts, and CSV export.

WHAT YOU'LL LEARN

- Infinite scroll with cursor pagination (`useInfiniteQuery`)
- Grouping by day with time zones (Africa/Addis_Ababa)

TASKS

- [ ] Activity list grouped by day, filters (kind, status, date range)
- [ ] Receipt page: reference (mono, copy), parties, amount, fee, timeline, share
- [ ] Download statement (CSV)

DONE WHEN

- [ ] Scrolling 500 items stays smooth; no duplicate rows

Day 35 · Sun 6 Dec — `test(web): Playwright end-to-end and accessibility checks`

Labels `test` `frontend` · **Milestone** M5 · **Branch** `test/day-35-playwright`

GITHUB ISSUE BODY

GOAL

The main journeys are tested in a real browser against the real API.

WHAT YOU'LL LEARN

- Playwright fixtures, test isolation, and seeded data
- axe-core accessibility scans in CI
- Running the full stack in CI with docker compose

TASKS

- [ ] Journeys: sign up → top up → send → see receipt; failed cash-out → refund visible
- [ ] axe scan on every main page; zero serious violations
- [ ] CI job with traces uploaded on failure
- [ ] Week 5 review: record a 60-second screen capture; journal entry

DONE WHEN

- [ ] E2E suite green in CI

Week 6 · M6 Requests, QR & admin

Day 36 · Mon 7 Dec — `feat(web): request money and the requests inbox`

Labels `frontend` · **Milestone** M6 Requests, QR & admin · **Branch** `feat/day-36-web-requests`

GITHUB ISSUE BODY

GOAL

Ask for money and handle incoming requests.

WHAT YOU'LL LEARN

- Badges and counts that stay in sync via query invalidation

TASKS

- [ ] Create request (recipient, amount, note)
- [ ] Inbox tabs: incoming / outgoing; pay (reuses PIN + review), decline, cancel
- [ ] Badge on Home for pending incoming requests

DONE WHEN

- [ ] Paying a request updates both users' views

Day 37 · Tue 8 Dec — `feat: merchant profiles and QR payments`

Labels `backend` `frontend` · **Milestone** M6 · **Branch** `feat/day-37-merchant-qr`

GITHUB ISSUE BODY

GOAL

Small businesses show a QR; customers scan and pay (FR-MER).

WHAT YOU'LL LEARN

- Signed QR payloads (HMAC) so codes can't be forged
- Camera access in the browser and graceful fallback

TASKS

- [ ] Backend: `merchants`, `POST /v1/merchants`, `GET /v1/merchants/me/qr`, `POST /v1/merchant-payments`
- [ ] Web: merchant QR screen, scanner (fallback: enter merchant code), pay flow
- [ ] Merchant fee rule

DONE WHEN

- [ ] Tampered QR payload is rejected

Day 38 · Wed 9 Dec — `feat(web): profile, security settings, and KYC upload`

Labels `frontend` `backend` `security` · **Milestone** M6 · **Branch** `feat/day-38-profile-kyc`

GITHUB ISSUE BODY

GOAL

Users manage their account and upgrade their tier.

WHAT YOU'LL LEARN

- Presigned uploads to S3-compatible storage (MinIO locally)
- Validating file type and size on both sides

TASKS

- [ ] Profile, change PIN, sessions list with revoke, dark mode toggle
- [ ] KYC: tier explanation, form, ID upload via presigned URL, status
- [ ] Backend presign endpoint; MinIO in compose

DONE WHEN

- [ ] Uploading a non-image file is rejected

Day 39 · Thu 10 Dec — `feat(admin): staff login with TOTP and the console shell`

Labels `admin` `security` · **Milestone** M6 · **Branch** `feat/day-39-admin-auth`

GITHUB ISSUE BODY

GOAL

A separate admin app and identity for staff, with MFA required.

WHAT YOU'LL LEARN

- Why staff and customers must never share an identity system
- TOTP (RFC 6238), recovery codes, and roles (support, compliance, finance, admin)

TASKS

- [ ] Backend: `staff_users`, `/admin/v1/auth/*`, separate signing key, role middleware
- [ ] `apps/admin`: login, TOTP enrolment, layout with sidebar
- [ ] CLI command to create the first admin

DONE WHEN

- [ ] A customer token is rejected by every admin route

Day 40 · Fri 11 Dec — `feat(admin): user search, KYC review, and wallet freeze`

Labels `admin` · **Milestone** M6 · **Branch** `feat/day-40-admin-users`

GITHUB ISSUE BODY

GOAL

Support and compliance can do their jobs.

WHAT YOU'LL LEARN

- Least-privilege role checks per action
- Confirm dialogs that state the exact effect, with a required reason

TASKS

- [ ] User search and detail (profile, wallets, recent transactions)
- [ ] KYC queue: approve / reject with reason → tier change
- [ ] Freeze / unfreeze (send-only or full) with reason; all audited

DONE WHEN

- [ ] Every action appears in `audit_logs` with before/after

Day 41 · Sat 12 Dec — `feat(admin): transaction explorer and integrity dashboard`

Labels `admin` `ledger` · **Milestone** M6 · **Branch** `feat/day-41-admin-explorer`

GITHUB ISSUE BODY

GOAL

See any transaction down to its journal lines; see that the books balance.

WHAT YOU'LL LEARN

- Presenting double-entry clearly to non-engineers (T-accounts)

TASKS

- [ ] Transaction search by reference, user, status, date
- [ ] Detail: events timeline, journal with debit/credit columns, provider payments
- [ ] Integrity page: latest drift and settlement reports, "run now"

DONE WHEN

- [ ] Every journal shown sums to zero on screen

Day 42 · Sun 13 Dec — `feat(admin): maker-checker adjustments and the audit log viewer`

Labels `admin` `ledger` `security` · **Milestone** M6 · **Branch** `feat/day-42-maker-checker`

GITHUB ISSUE BODY

GOAL

Manual corrections need two different people and leave a permanent trail.

WHAT YOU'LL LEARN

- Four-eyes principle (maker-checker) and segregation of duties
- Why adjustments are journals, never balance edits

TASKS

- [ ] Backend: `adjustments` create → approve/reject; approver ≠ creator; approval posts a journal
- [ ] Admin UI: create (balanced lines, reason, evidence), pending queue, approve/reject
- [ ] Audit log viewer with filters
- [ ] Week 6 review; journal entry

DONE WHEN

- [ ] Test: the creator cannot approve their own adjustment

Week 7 · M7 Hardening & launch

Day 43 · Mon 14 Dec — `feat: risk rules engine — allow, flag, hold, block`

Labels `backend` `security` · **Milestone** M7 Hardening & launch · **Branch** `feat/day-43-risk-engine`

GITHUB ISSUE BODY

GOAL

Every money movement is scored by simple, explainable rules (PRD §12.2).

WHAT YOU'LL LEARN

- Rules engines vs ML for early-stage fraud
- Velocity checks with SQL windows
- Fail-safe defaults: what happens when the risk check errors

TASKS

- [ ] `risk.Evaluate` with rules: velocity, new-device large amount, new recipient large amount, rapid in-out
- [ ] `risk_decisions`, `risk_flags`; hold keeps funds reserved
- [ ] Rules configurable; every decision stores which rules fired

DONE WHEN

- [ ] Table tests for each rule and the combined outcome

Day 44 · Tue 15 Dec — `feat(admin): risk flags queue`

Labels `admin` · **Milestone** M7 · **Branch** `feat/day-44-risk-flags-ui`

GITHUB ISSUE BODY

GOAL

Analysts review held payments and release or reverse them.

TASKS

- [ ] Queue with filters; detail shows fired rules and user history
- [ ] Release (completes the payment) or reject (refund), with notes; audited
- [ ] Customer sees "needs a quick review" copy, never "fraud"

DONE WHEN

- [ ] Releasing a held transfer completes it exactly once

Day 45 · Wed 16 Dec — `feat: observability with OpenTelemetry, Prometheus, and Grafana`

Labels `backend` `infra` · **Milestone** M7 · **Branch** `feat/day-45-observability`

GITHUB ISSUE BODY

GOAL

You can answer "is it healthy?" and "why was that slow?" (PRD §19).

WHAT YOU'LL LEARN

- RED metrics (rate, errors, duration) and business metrics
- Distributed tracing across API, DB, worker
- Writing alerts and runbooks

TASKS

- [ ] OTel traces (Gin, pgx, River); trace ID in logs
- [ ] Metrics: HTTP, transfers by outcome, drift accounts, outbox lag, webhook failures
- [ ] Prometheus + Grafana + Tempo in `docker-compose.observability.yml`; dashboard JSON committed
- [ ] `docs/runbooks/` for drift detected, outbox lag, provider down

DONE WHEN

- [ ] Screenshot of the dashboard in the PR

Day 46 · Thu 17 Dec — `perf: k6 load test and tuning report`

Labels `test` `infra` · **Milestone** M7 · **Branch** `perf/day-46-load-test`

GITHUB ISSUE BODY

GOAL

Measure against the NFRs (p95 < 150 ms at 200 rps; 500 transfers/s).

WHAT YOU'LL LEARN

- Load test design: realistic mix, ramp-up, think time
- Reading p50/p95/p99 and finding the bottleneck (pool size, indexes, lock waits)

TASKS

- [ ] k6 scripts: browse mix and transfer storm between 1,000 seeded wallets
- [ ] Run, profile (pprof, `pg_stat_statements`), fix the top issue, run again
- [ ] `docs/perf.md` with before/after numbers and graphs
- [ ] Drift monitor reports zero after the storm

DONE WHEN

- [ ] Report states honestly whether each NFR is met

Day 47 · Fri 18 Dec — `security: threat model and ASVS level 1 review`

Labels `security` · **Milestone** M7 · **Branch** `security/day-47-asvs-review`

GITHUB ISSUE BODY

GOAL

A written, checkable security posture.

WHAT YOU'LL LEARN

- STRIDE threat modelling on your own architecture
- OWASP ASVS as a checklist; dependency and image scanning

TASKS

- [ ] `docs/security/threat-model.md` (STRIDE per component)
- [ ] `docs/security/asvs-l1.md` checklist with evidence links
- [ ] govulncheck, `pnpm audit`, Trivy image scan in CI
- [ ] Security headers (CSP, HSTS) on web apps; fix findings

DONE WHEN

- [ ] No high/critical findings open without a written reason

Day 48 · Sat 19 Dec — `chore: production images, deployment, and demo seed data`

Labels `infra` · **Milestone** M7 · **Branch** `chore/day-48-deploy`

GITHUB ISSUE BODY

GOAL

A public demo anyone can try in two minutes.

WHAT YOU'LL LEARN

- Multi-stage Docker builds, distroless images, non-root users
- Running migrations safely on deploy
- Secrets management on a hosting platform

TASKS

- [ ] Dockerfiles for api, worker, simulator, web, admin
- [ ] Deploy (Fly.io or Railway for Go, Vercel for Next.js, managed Postgres)
- [ ] `make seed-demo`: demo users, merchants, a month of history
- [ ] Demo banner: "Simulated money. Twofold is a portfolio project."

DONE WHEN

- [ ] Fresh visitor can sign up with a demo OTP and send money

Day 49 · Sun 20 Dec — `docs: launch — case study, ADR index, and demo video`

Labels `docs` · **Milestone** M7 · **Branch** `docs/day-49-launch`

GITHUB ISSUE BODY

GOAL

Make the work easy to understand in five minutes.

WHAT YOU'LL LEARN

- Writing for reviewers: lead with outcomes, show evidence
- Telling the engineering story in interviews

TASKS

- [ ] README: hero screenshot, live demo link, architecture diagram, "how we know it's correct" (property, concurrency, drift, BOLA)
- [ ] `docs/case-study.md` : problem, decisions, trade-offs, numbers, what you'd do next
- [ ] ADR index; every ADR in your own words
- [ ] 3-minute demo video; LinkedIn post draft
- [ ] Final journal entry: what the 49 days taught you

DONE WHEN

- [ ] A friend can explain what Twofold does after reading only the README